

[Linux] Cockpit이란? Linux 서버를 웹에서 관리하는 방법

SSH로 Linux 서버에 접속하다 보면 로그인 직후 다음과 같은 안내 문구를 볼 때가 있다.

```
Activate the web console with: systemctl enable --now cockpit.socket
```

처음 보면 별도의 필수 설정처럼 보이지만, 이것은 오류 메시지가 아니다. **Cockpit**이라는 웹 기반 Linux 서버 관리 콘솔을 사용하고 싶다면 활성화할 수 있다는 안내다.

Cockpit이란?

Cockpit은 Linux 서버를 웹 브라우저에서 관리할 수 있게 해주는 오픈소스 웹 관리 도구다.

일반적으로 Linux 서버는 SSH로 접속한 뒤 명령어로 관리한다.

```
내 PC
|
| SSH : 22
▼
Linux Server
```

Cockpit을 활성화하면 여기에 웹 관리 화면이 하나 더 생긴다.

```
내 PC의 웹 브라우저
|
| HTTPS : 9090
▼
Cockpit
|
▼
Linux Server
```

즉, Cockpit은 Linux를 대신하는 별도의 운영체제나 클라우드 관리 콘솔이 아니다. **현재 서버에서 실행되면서 CPU, 메모리, 네트워크, 서비스, 로그 등의 정보를 웹 UI로 보여주고 일부 관리 작업을 수행할 수 있게 해주는 도구다.**

Cockpit 화면에서는 배포판과 설치된 구성에 따라 다음과 같은 작업을 할 수 있다.

- CPU, 메모리, 디스크 등 서버 리소스 사용량 확인
- 시스템 로그 확인
- 네트워크 인터페이스 및 네트워크 상태 확인
- 사용자 계정 확인 및 관리
- systemd 서비스 상태 확인 및 시작·중지
- 소프트웨어 업데이트 확인
- 웹 터미널 사용

SSH 명령어가 익숙하지 않은 사람에게는 서버 상태를 직관적으로 확인할 수 있다는 장점이 있고, SSH에 익숙한 경우에도 **상태 확인용 보조 관리 화면**으로 활용할 수 있다.

cockpit.socket은 무엇일까?

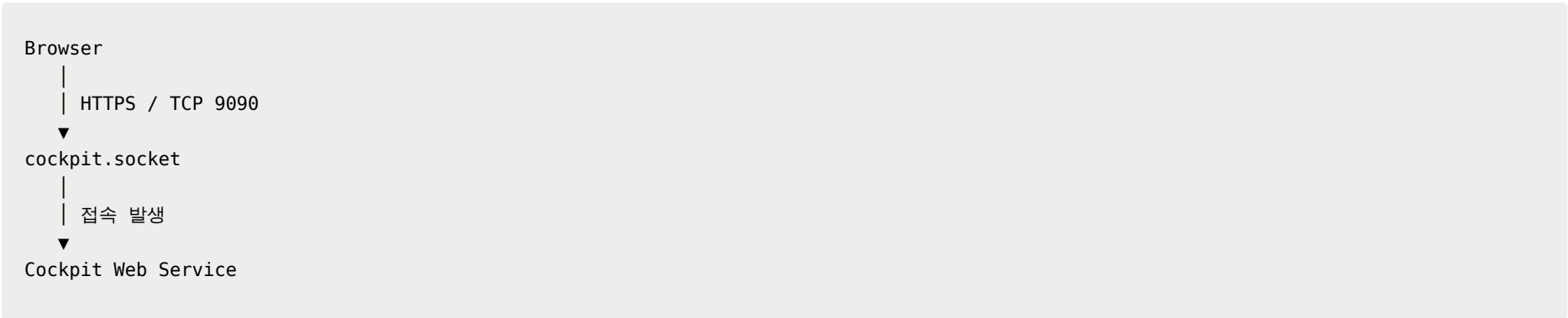
SSH 로그인 화면에서 안내하는 명령어는 다음과 같다.

```
sudo systemctl enable --now cockpit.socket
```

여기서 중요한 부분은 `cockpit.service` 가 아니라 `cockpit.socket` 이라는 점이다.

Cockpit은 systemd의 **Socket Activation** 방식을 사용할 수 있다. 항상 Cockpit 웹 서비스를 실행해 두는 대신 `cockpit.socket` 이 특정 포트로 들어오는 연결을 기다리고 있다가 실제 접속이 발생하면 필요한 서비스를 실행하는 방식이다.

기본적으로 Cockpit은 **TCP 9090 포트**를 사용한다.



따라서 SSH만 사용할 예정이라면 Cockpit을 반드시 활성화할 필요는 없다.

Rocky Linux에서 Cockpit 활성화해보기

이번에는 Rocky Linux 9 계열 서버에서 Cockpit을 직접 활성화하고 브라우저로 접속해봤다.

1. 현재 Cockpit 상태 확인

먼저 `cockpit.socket` 의 상태를 확인한다.

```
systemctl status cockpit.socket
```

아직 활성화하지 않았다면 다음과 같이 `disabled` , `inactive (dead)` 상태를 확인할 수 있다.

```
[soojin@hsj-public-svr ~]$ systemctl status cockpit.socket
○ cockpit.socket - Cockpit Web Service Socket
   Loaded: loaded (/usr/lib/systemd/system/cockpit.socket; disabled; preset: disabled)
   Active: inactive (dead)
   Triggers: ● cockpit.service
   Docs: man:cockpit-ws(8).
   Listen: [::]:9090 (Stream)
[soojin@hsj-public-svr ~]$
```

여기서 `disabled` 와 `inactive` 는 의미가 조금 다르다.

- **disabled**: 부팅 시 자동 활성화되도록 등록되어 있지 않음
- **inactive (dead)**: 현재 실행 중이 아님

2. cockpit.socket 활성화

Cockpit을 바로 사용하면서, 서버 재부팅 이후에도 사용할 수 있도록 다음 명령어를 실행한다.

```
sudo systemctl enable --now cockpit.socket
```

```
[soojin@hsj-public-svr ~]$ sudo systemctl enable --now cockpit.socket
[sudo] password for soojin:
[soojin@hsj-public-svr ~]$
```

`enable` 과 `--now` 를 함께 사용했기 때문에 두 가지 작업이 동시에 수행된다.

- `enable` : 다음 부팅 이후에도 `cockpit.socket` 이 활성화되도록 설정
- `--now` : 재부팅을 기다리지 않고 지금 바로 활성화

즉 다음 두 작업을 한 번에 수행하는 것과 비슷하다.

```
sudo systemctl enable cockpit.socket
sudo systemctl start cockpit.socket
```

9090 포트 접근 허용

Cockpit을 활성화했다라도 외부 PC의 브라우저에서 서버까지 TCP 9090 통신이 도달할 수 있어야 한다.

클라우드 서버라면 서버 앞단의 방화벽 또는 Security Group, ACG와 같은 접근 제어 정책에서 **TCP 9090 포트**를 허용해야 한다.

예를 들어 Naver Cloud Platform 서버라면 서버에 적용된 ACG에서 테스트를 수행할 출발지 IP에 대해 TCP 9090을 허용할 수 있다.



9090 포트를 인터넷 전체에 무조건 공개하기보다는 **관리자 IP 등 필요한 출발지만 허용하는 방식**이 안전하다. Cockpit은 서버 관리 기능을 제공하므로 접근 범위를 최소화하는 것이 좋다.

서버 내부에서 `firewalld`를 사용하고 있다면 OS 방화벽에서도 Cockpit 접근이 허용되어 있어야 한다. Rocky Linux에서는 환경에 따라 다음과 같이 Cockpit 서비스를 허용할 수 있다.

```
sudo firewall-cmd --permanent --add-service=cockpit
sudo firewall-cmd --reload
```

브라우저에서 Cockpit 접속

Cockpit의 기본 접속 주소는 다음과 같다.

```
https://SERVER_IP:9090
```

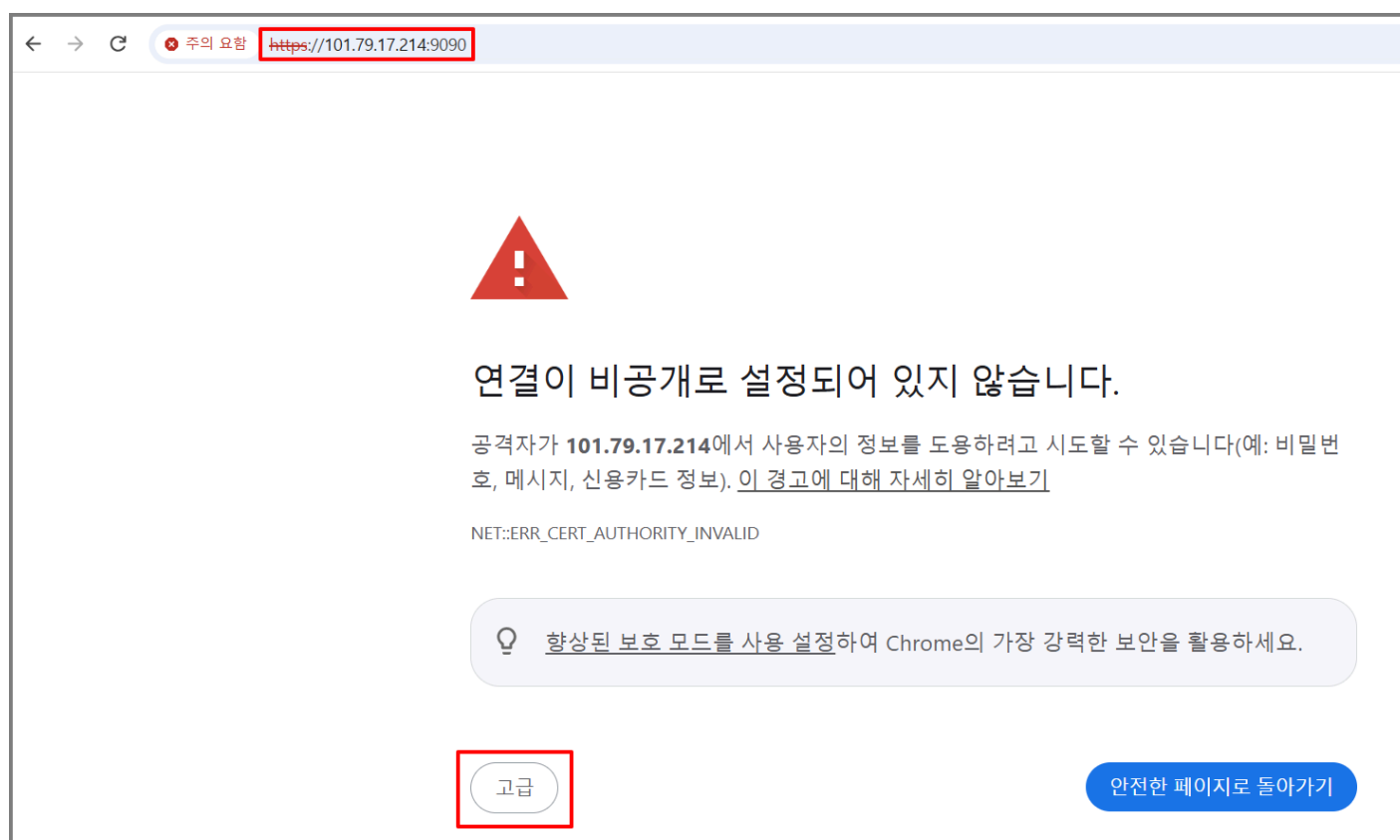
예를 들어 서버 IP가 `203.0.113.10` 이라면 다음과 같이 접속한다.

```
https://203.0.113.10:9090
```

HTTP가 아니라 **HTTPS**를 사용한다는 점도 확인해야 한다.

인증서 경고가 나타나는 이유

처음 접속하면 브라우저에서 인증서 경고가 나타날 수 있다.

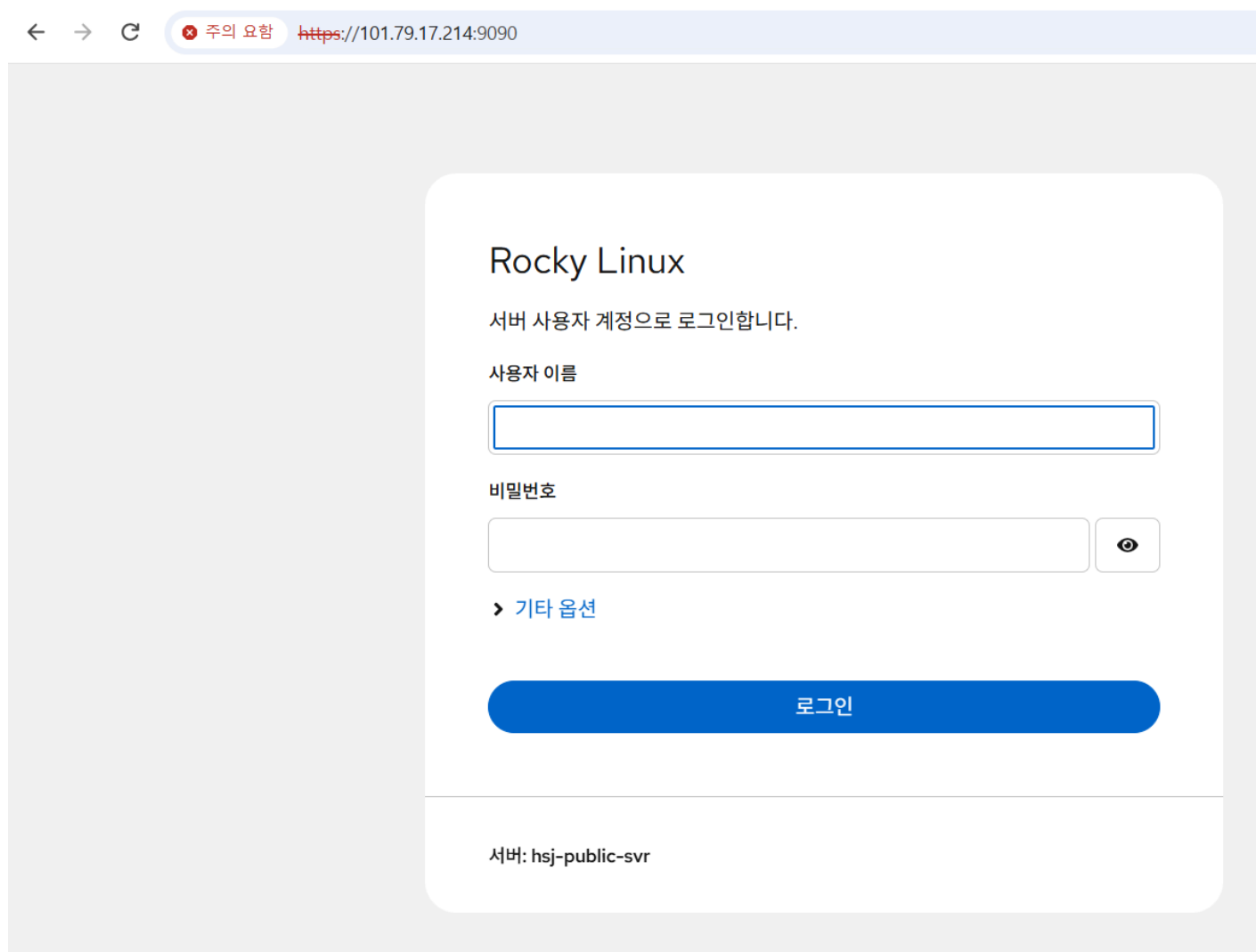


Cockpit이 기본적으로 제공하는 인증서가 공인 인증기관에서 발급받아 브라우저가 신뢰하는 인증서가 아니거나, 접속한 IP 주소와 인증서 이름이 일치하지 않으면 브라우저는 해당 연결을 자동으로 신뢰하지 않는다.

따라서 실습 환경에서는 경고 내용을 확인한 뒤 접속을 진행할 수 있지만, 실제 운영 환경에서 외부에 제공한다면 인증서 구성과 접근 제어를 별도로 검토해야 한다.

Cockpit 로그인

접속에 성공하면 Rocky Linux 로그인 화면이 나타난다.



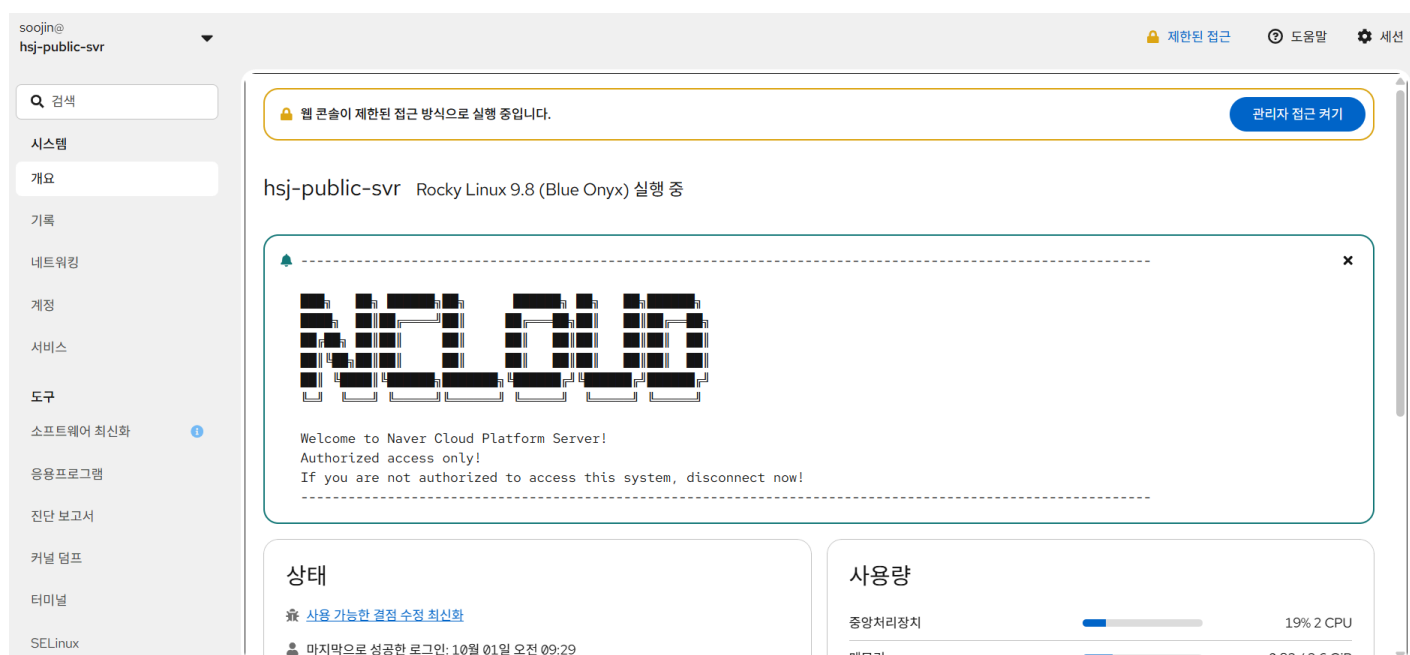
여기에는 별도의 Cockpit 전용 계정을 만드는 것이 아니라 **서버에 존재하는 Linux 사용자 계정**으로 로그인한다.

즉 SSH에서 사용하는 것과 동일한 서버 사용자 계정을 기반으로 인증한다.

다만 로그인한 사용자가 서버에서 가지고 있는 권한에 따라 Cockpit에서 수행할 수 있는 작업도 달라진다.

Cockpit 관리 화면

로그인하면 서버의 개요 화면을 확인할 수 있다.



대시보드에서는 서버의 운영체제 정보와 CPU·메모리 등의 상태를 확인할 수 있고, 왼쪽 메뉴를 통해 로그, 네트워크, 계정, 서비스, 터미널 등의 기능에 접근할 수 있다.

특히 웹 화면 안에 터미널도 제공되기 때문에 필요한 경우 브라우저에서 바로 명령어를 실행할 수도 있다.

다만 Cockpit이 있다고 해서 SSH가 필요 없어지는 것은 아니다. 서버 장애나 네트워크 문제, Cockpit 자체가 정상적으로 동작하지 않는 상황에서는 여전히 SSH와 콘솔 접속이 중요하다.

Cockpit과 SSH는 어떤 관계일까?

둘은 경쟁 관계라기보다 목적이 조금 다르다.

구분	SSH	Cockpit
접속 방식	터미널	웹 브라우저
대표 포트	TCP 22	TCP 9090
조작 방식	CLI 명령어	GUI + 웹 터미널
장점	세밀한 제어와 자동화에 유리	서버 상태를 직관적으로 확인하기 편함

서버 운영에 익숙해질수록 SSH와 CLI가 핵심 도구가 되지만, Cockpit은 **Linux 서버에서 실제로 어떤 서비스와 리소스가 동작하고 있는지를 시각적으로 확인하는 도구**로 꽤 유용하다.

사용하지 않는다면?

SSH 로그인 시 다음 안내가 표시되더라도 Cockpit을 사용할 계획이 없다면 아무 작업도 하지 않아도 된다.

```
Activate the web console with: systemctl enable --now cockpit.socket
```

이 문구는 단순히 "웹 관리 콘솔을 사용하고 싶다면 Cockpit을 활성화할 수 있다"는 안내일 뿐이다.

이미 활성화한 Cockpit을 다시 사용하지 않으려면 다음과 같이 비활성화할 수 있다.

```
sudo systemctl disable --now cockpit.socket
```

정리하면 Cockpit은 Linux 서버를 브라우저에서 관리할 수 있게 해주는 웹 콘솔이며, Rocky Linux에서는 `cockpit.socket` 을 활성화한 뒤 기본적으로 HTTPS 9090 포트를 통해 접속할 수 있다. SSH를 대체하기보다는 서버 상태 확인과 관리 작업을 좀 더 직관적으로 만들어주는 보조 도구라고 이해하면 된다.